

Introducción a la programación RTL en C++ para simulación de procesamientos digitales de señales

Ing. Leonardo Luis Ortiz

Instituto Balseiro
Centro Atómico Bariloche

- El dictado del curso se orienta a proveer al alumno de la capacidad de diseñar sistemas de procesamiento de señal, y utilizar el simulador HALCON como herramienta para la verificación y análisis.

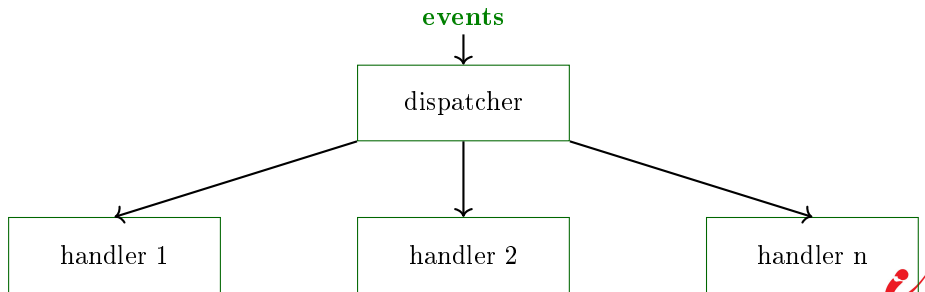
- 1 Introducción
- 2 Componentes de un Sistema Basado en Eventos
- 3 Flujo de Eventos y Gestión de Eventos
- 4 Arquitectura Basada en Eventos (EDA)
- 5 Modelos de Concurrencia Basados en Eventos

Introducción

Introducción

La Programación controlada por Eventos (Event-driven Programming - EDP) es un paradigma donde el flujo de un programa está determinado por eventos, en lugar de una secuencia predefinida de instrucciones. Los eventos pueden activarse mediante interacciones del usuario, señales del sistema o fuentes externas.

“Un evento es un hecho inmutable, algo que ha sucedido en el pasado y que no se puede cambiar”.



Ciclo de vida de EDP:

- **Generación de eventos:** Los eventos pueden generarse mediante la entrada del usuario, procesos en segundo plano, señales de hardware o comunicaciones externas. Algunos ejemplos incluyen hacer clic en un botón, recibir una solicitud HTTP o detectar movimiento de un sensor.
- **Detección de eventos:** Un detector de eventos monitoriza eventos específicos y notifica al sistema cuando ocurren. Este mecanismo garantiza una respuesta en tiempo real.
- **Gestión de eventos:** Una vez detectado un evento, el controlador correspondiente ejecuta una función predefinida para procesarlo. Esto puede implicar actualizar una interfaz de usuario, procesar datos o enviar solicitudes de red.

- **Ejecución asíncrona:** a diferencia de la programación secuencial tradicional, la EDP permite que los controladores de eventos se ejecuten de forma independiente sin bloquear otras operaciones.
- **Desacoplamiento de componentes:** los eventos facilitan el diseño modular, lo que permite que las diferentes partes de una aplicación interactúen sin dependencias directas.
- **Funciones de callbacks:** los handlers de eventos suelen implementarse como callbacks, que se invocan cuando ocurre un evento específico.
- **Lógica basada en el estado (State-Driven Logic):** el comportamiento de la aplicación se determina por la ocurrencia de eventos, en lugar de un flujo de control fijo.

Event-Driven Programming - Ventajas

- **Ejecución Asíncrona:** A diferencia de la ejecución secuencial, los sistemas basados en eventos pueden gestionar múltiples tareas simultáneamente, lo que mejora la capacidad de respuesta. Esto es fundamental en aplicaciones como servidores web, donde varios usuarios interactúan simultáneamente.
- **Escalabilidad mejorada:** EDP permite gestionar eventos de gran volumen de forma eficiente, lo que la hace ideal para microservicios, arquitecturas sin servidor y aplicaciones en tiempo real.
- **Diseño modular y extensible:** Los eventos desacoplan los componentes, lo que permite un código flexible y reutilizable. Los desarrolladores pueden modificar los controladores de eventos de forma independiente sin afectar a todo el sistema.
- **Tiempo de inactividad de la CPU reducido:** Dado que los loops de eventos monitorizan y distribuyen eventos continuamente, los recursos de la CPU se utilizan eficazmente, lo que hace que EDP sea ideal para aplicaciones de baja latencia.

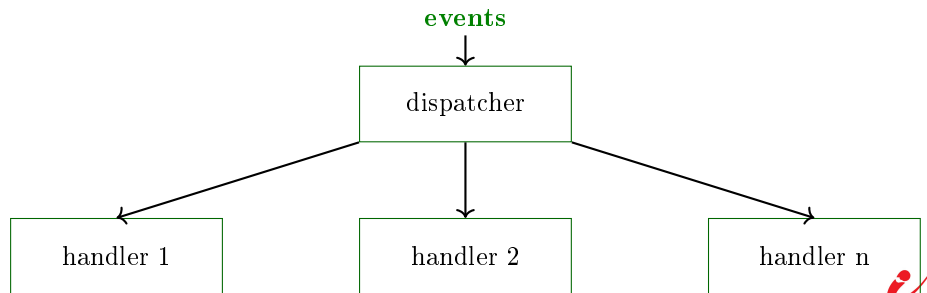
- **Difícil depuración y testeo:** La ejecución asíncrona dificulta el seguimiento de secuencias de eventos y la reproducción de problemas. Las herramientas de depuración, como los registros de eventos (event logs) y los frameworks de seguimiento, son esenciales.
- **Problemas con las callbacks:** Anidar múltiples devoluciones de llamadas puede generar código espagueti, lo que reduce la legibilidad y el mantenimiento. Las promesas y los patrones `async/await` mitigan este problema.
- **Complejidad en la Gestión del Estado:** Gestionar múltiples eventos simultáneos requiere una sincronización cuidadosa para evitar condiciones de carrera e inconsistencias en los datos.
- **Consumo de Memoria:** Los escuchas de eventos de larga duración pueden consumir memoria con el tiempo, lo que puede provocar fugas de memoria si no se gestionan adecuadamente.

- GUI applications (Graphical User Interface)
- Cloud-base microservices
- IoT devices
- Real Time Systems (RTS)
- Networking
- Distributed Systems

Componentes de un Sistema Basado en Eventos

Event-Driven Programming

Un sistema basado en eventos consta de múltiples componentes que trabajan en conjunto para garantizar un procesamiento eficiente de eventos. Los elementos clave incluyen **productores de eventos** y **consumidores de eventos**, que generan y procesan eventos; **event loops** y **event handlers**, que gestionan la ejecución asíncrona; **event listeners** y **dispatchers**, que detectan y distribuyen los eventos; y **middleware**, que facilita la comunicación y el flujo de eventos.



Modelo Event Producers & Consumers

En una arquitectura basada en eventos, los **productores de eventos** generan eventos, mientras que los **consumidores** los procesan. Los productores pueden incluir interacciones del usuario (clics del ratón, pulsaciones de teclas), eventos del sistema (actualizaciones de archivos, solicitudes de red) o fuentes externas (llamadas a API, sensores IoT). Los consumidores son responsables de gestionar estos eventos y ejecutar las acciones pertinentes.

Los productores y consumidores de eventos operan de forma **independiente**, lo que hace que los sistemas basados en eventos sean altamente escalables y estén débilmente acoplados. Esta disociación permite a los productores enviar eventos sin saber cómo ni cuándo los procesarán los consumidores. Para optimizar la comunicación, los sistemas suelen utilizar colas de mensajes (**event queues**) o **event brokers** para gestionar el flujo de eventos. Entre los ejemplos de productores de eventos se incluyen las interfaces de usuario, los servidores web y los sistemas de monitorización en tiempo real, mientras que los consumidores pueden ser servicios de registro, gestores de notificaciones o tareas en segundo plano.

Modelo Event Producers & Consumers



Modelo Event Producers & Consumers: Producers

Los productores de eventos inician el flujo de eventos emitiendo señales para notificar cambios al sistema. Se pueden clasificar en:

- **Producers generados por el usuario** : aplicaciones GUI donde los usuarios interactúan mediante botones, pulsaciones de teclas o gestos.
- **Producers generados por el sistema** : aplicaciones GUI donde los usuarios interactúan mediante botones, pulsaciones de teclas o gestos.
- **Producers externos** : webhooks, API, dispositivos IoT o servicios en la nube que activan flujos de trabajo basados en eventos.

Los **Producers** no suelen esperar la respuesta de los **Consumers**. En su lugar, publican eventos en un **bus de eventos**, **cola de mensajes** o un **broker**, lo que garantiza la escalabilidad y la ejecución asincrónica.

- **Consumer único** : Un consumidor procesa un tipo de evento específico (p. ej., un observador de archivos que actualiza una base de datos).
- **Múltiples consumers** : Varios controladores procesan el mismo evento (p. ej., una notificación por correo electrónico y un sistema de registro que responden a un evento de registro de usuario).
- **Event pipelines** : Los eventos disparan procesos secuenciales (p. ej., un sensor de IoT que envía datos → se almacenan en una base de datos → se analizan en tiempo real).

Los consumers utilizan controladores basados en eventos para ejecutar tareas al recibir un evento, lo que permite que el sistema responda a los cambios en tiempo real.

Los loops y handlers (controladores) de eventos son componentes esenciales de la programación basada en eventos, lo que permite la ejecución asíncrona y un procesamiento eficiente de eventos. El loop de eventos es un ciclo continuo que detecta los eventos entrantes y los envía a controladores registrados para su ejecución. Los handlers de eventos son funciones o callbacks que procesan eventos específicos, garantizando la capacidad de respuesta de la aplicación. Estos componentes son esenciales en interfaces gráficas de usuario (GUI), servidores web, aplicaciones de red y sistemas en tiempo real.

Event loops & Handlers: Event Loop

- **Event Detection:** Monitorea los eventos entrantes desde colas, sockets o listeners.
- **Event Dispatching:** Asigna los eventos detectados a los handlers correspondientes.
- **Event Execution:** Procesa el evento mediante el handler correspondiente.
- **Idle State:** Espera nuevos eventos cuando no hay ninguno pendiente.

Al gestionar múltiples eventos de forma asincrónica, los bucles de eventos evitan que las aplicaciones se bloqueen en tareas individuales, lo que mejora significativamente la eficiencia y la escalabilidad.

Un controlador de eventos es una función que se ejecuta cuando ocurre un evento específico. Los controladores deben ser no bloqueantes para evitar cuellos de botella en el rendimiento del loop de eventos. Pueden ser:

- **Controladores síncronos:** se ejecutan secuencialmente, esperando a que se complete la ejecución antes de procesar el siguiente evento.
- **Controladores asíncronos:** utilizan operaciones no bloqueantes (por ejemplo, `async/await` en Python) para gestionar varios eventos simultáneamente.

Un manejo eficiente de eventos evita **race conditions**, **callback hell** y **degradación del rendimiento**.

Los **listeners** y **dispatchers** de eventos son fundamentales para las arquitecturas basadas en eventos, ya que permiten la detección y distribución eficiente de eventos. Un **listener** de eventos monitoriza eventos específicos y reacciona cuando ocurren, mientras que un **dispatcher** de eventos garantiza que el evento se enrute al controlador correcto. Estos componentes permiten que las aplicaciones sean responsivas, modulares y escalables al desvincular a los productores de eventos de los consumers.

Algunos casos de uso comunes incluyen:

- **Aplicaciones GUI:** Los listeners detectan clics de botones, movimientos del ratón y pulsado de teclas.
- **Aplicaciones web:** monitorizan solicitudes HTTP, conexiones WebSocket y actualizaciones en la base de datos.
- **Sistemas IoT:** Los dispositivos “escuchan” actualizaciones de datos de sensores o señales externas.

Los Listeners registran eventos específicos y ejecutan las callbacks asignadas cuando ocurren. Pueden ser síncronos o asíncronos, según los requisitos del sistema.

Event Listeners & Dispatchers: Dispatchers

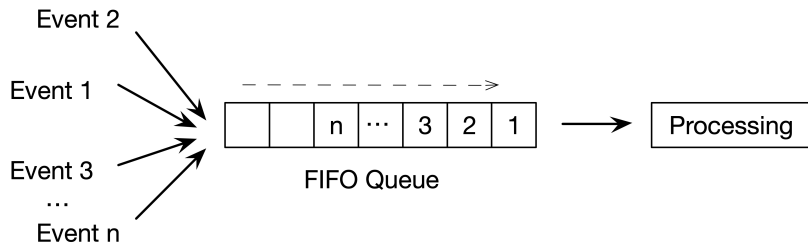
Un dispatcher de eventos dirige los eventos detectados a los controladores adecuados. Garantiza que los eventos lleguen a los componentes correctos, lo que permite un procesamiento modular de eventos.

Los despachadores pueden operar en diferentes modos:

- **Direct Dispatching:** los eventos se envían directamente a un único controlador predefinido.
- **Broadcasting:** los eventos se envían a múltiples suscriptores (modelo publish-subscribe).
- **Queue-Based Dispatching:** los eventos se ponen en cola y se procesan en orden.

Un despacho eficiente optimiza el rendimiento al garantizar que los eventos no bloqueen la ejecución mientras esperan su procesamiento.

Event Listeners & Dispatchers: Dispatchers



El **middleware** en sistemas basados en eventos actúa como una capa intermedia que facilita la comunicación, el procesamiento de eventos y el enrutamiento de mensajes entre los producers y los consumers de eventos. Proporciona servicios esenciales como el filtrado de eventos, transformaciones, logging, seguridad y la gestión de colas de mensajes, garantizando la escalabilidad, la fiabilidad y el desacoplamiento en arquitecturas distribuidas. El middleware se utiliza ampliamente en microservicios, aplicaciones en tiempo real y sistemas empresariales basados en eventos para gestionar eficientemente flujos de eventos complejos.

- RabbitMQ
- Apache Kafka
- Redis Pub/Sub

El middleware mejora los sistemas basados en eventos gestionando el pre-procesamiento y el pos-procesamiento de eventos. Algunas de sus responsabilidades clave son:

- **Filtrado de eventos:** Selecciona eventos relevantes según reglas predefinidas, evitando el procesamiento innecesario.
- **Transformación de eventos:** Convierte formatos de eventos para garantizar la compatibilidad entre servicios (p. ej., JSON a XML).
- **Enrutamiento de mensajes:** Dirige los eventos a los consumers adecuados en arquitecturas de publicación-suscripción.
- **Seguridad y autenticación:** Garantiza la transmisión segura de eventos entre componentes. item **Logging y monitoreo:** Rastrea los flujos de eventos para su depuración y análisis.

El middleware garantiza el correcto funcionamiento de las arquitecturas basadas en eventos en entornos distribuidos, estandarizando la transmisión de eventos y reduciendo la complejidad del sistema.

Flujo de Eventos y Gestión de Eventos

El flujo y la gestión de eventos definen cómo se propagan los eventos a través de una aplicación y cómo se procesan. Un flujo de eventos estructurado garantiza una gestión de eventos eficiente y organizada, evitando conflictos y operaciones redundantes. Comprender la propagación de eventos, los mecanismos de gestión y la secuenciación es crucial para diseñar sistemas basados en eventos escalables y con capacidad de respuesta en aplicaciones GUI, desarrollo web y computación distribuida.

Propagación de eventos

La propagación de eventos determina cómo se mueven los eventos a través de la jerarquía de componentes de una aplicación. En interfaces gráficas de usuario (GUI), aplicaciones web y sistemas basados en mensajes, los eventos viajan a través de múltiples elementos antes de ser procesados. La propagación sigue una secuencia estructurada, lo que garantiza que los oyentes de eventos en diferentes niveles puedan responder adecuadamente. Los dos modelos de propagación principales son:

- **De arriba a abajo (fase de captura):** Los eventos viajan desde el elemento raíz hasta el elemento de destino.
- **De abajo a arriba (fase de propagación):** Los eventos se originan en el elemento de destino y se propagan hacia arriba.

Controlar la propagación de eventos ayuda a los desarrolladores a gestionar conflictos de eventos, optimizar el rendimiento e implementar comportamientos complejos basados en eventos, como la delegación y la interceptación.

Los sistemas basados en eventos pueden gestionar eventos de forma síncrona o asíncrona, según los requisitos de rendimiento y capacidad de respuesta.

- **Synchronous Handling:** Los eventos se procesan secuencialmente, bloqueando la ejecución hasta que el handler finaliza. Esto garantiza un orden predecible, pero puede causar retrasos en aplicaciones con alto tráfico.
- **Asynchronous Handling:** Los eventos se ejecutan simultáneamente, lo que evita bloqueos y mejora la capacidad de respuesta. Esto es fundamental para las operaciones de red, las aplicaciones en tiempo real y los sistemas basados en eventos a gran escala.

La elección entre modelos síncronos y asíncronos depende de factores como las dependencias de los eventos, el tiempo de ejecución y las necesidades de concurrencia. Una gestión eficiente de eventos mejora la experiencia del usuario y el rendimiento de las aplicaciones.

Control de Eventos: Síncronos & Asíncronos

Característica	Síncrono	Asíncrono
Orden de ejecución	Secuencial	Concurrente
Conducta	Bloqueante	No bloqueante
Performance	Lento	Rápido
Complejidad	Fácil	Más complejo
Casos de uso	Tareas simples, aplicaciones de baja latencia	Aplicaciones de alta performance, operaciones pesadas de E/S

Gestión de Secuencias de Eventos y Dependencias

En aplicaciones complejas basadas en eventos, varios eventos ocurren en secuencia, a veces con dependencias entre ellos. Gestionar las secuencias de eventos garantiza un orden de ejecución correcto y evita conflictos.

Las estrategias clave para gestionar las dependencias de eventos incluyen:

- **Event Queues:** organizan eventos en una secuencia estructurada para evitar conflictos.
- **Callbacks & Promises:** garantizan que los eventos dependientes se ejecuten en el orden correcto.
- **Event Prioritization:** asignan niveles de prioridad a eventos críticos.

Una secuencia de eventos bien gestionada previene condiciones de carrera, deadlocks (bloqueos) y comportamientos imprevistos, lo que garantiza un rendimiento fluido de la aplicación. El flujo y la gestión de eventos definen cómo responden las aplicaciones a las acciones del usuario y a los eventos del sistema. Un control de propagación adecuado, estrategias de gestión y gestión de dependencias permiten a los desarrolladores crear arquitecturas basadas en eventos eficientes, ágiles y fáciles de mantener.

La secuenciación de eventos define el orden en que se ejecutan para mantener un flujo lógico. Algunos eventos pueden necesitar ejecutarse antes que otros debido a dependencias de datos, reglas de negocio o restricciones del sistema.

Estrategias para la secuenciación de eventos:

- **Ordenación manual:** Defina reglas explícitas para ejecutar eventos en una secuencia fija.
- **Ejecución basada en prioridad:** Asignar niveles de prioridad a los eventos para garantizar que las tareas de mayor prioridad se ejecuten primero.
- **Ejecución basada en estado:** Ejecutar eventos solo cuando un sistema alcanza un estado específico (por ejemplo, esperar la entrada del usuario antes de enviar un formulario).

Algunos eventos no pueden ejecutarse hasta que se completen los eventos prerequisite. Las dependencias deben gestionarse con cuidado para evitar interbloqueos, condiciones de carrera y transiciones de estado inconsistentes.

Tipos de dependencias de eventos:

- **Dependencias rígidas:** Un evento debe completarse antes de que comience otro. Ejemplo: Una actualización de la base de datos debe finalizar antes de enviar un correo electrónico de confirmación.
- **Dependencias blandas:** Algunos eventos pueden ejecutarse en paralelo, pero puede ser necesaria la sincronización. Ejemplo: Varios microservicios procesan solicitudes de usuario simultáneamente.

Arquitectura Basada en Eventos (EDA)

Event-Driven Architecture (EDA)

La Arquitectura Basada en Eventos (EDA) es un paradigma de diseño de software donde los componentes del sistema interactúan principalmente mediante la generación, detección y respuesta a eventos. A diferencia de los modelos tradicionales de solicitud-respuesta, la EDA permite la comunicación asíncrona, alta escalabilidad y flexibilidad, lo que la hace ideal para aplicaciones en tiempo real, microservicios y sistemas distribuidos.

Principios de la Arquitectura Basada en Eventos

La EDA se basa en un enfoque descentralizado y reactivo, donde los sistemas responden dinámicamente a los eventos en lugar de seguir un flujo de trabajo rígido.

- **Event Producers & Consumers:** Componentes que generan y responden a eventos de forma independiente.
- **Brokers de Eventos y Middleware:** Facilitan la transmisión de eventos entre componentes desacoplados.
- **Procesamiento Asíncrono:** Los eventos se gestionan sin bloquear otros procesos, lo que mejora la eficiencia.
- **Escalabilidad y tolerancia a fallos:** la gestión independiente de eventos permite el escalamiento horizontal y la resiliencia.

Event-Driven Architecture (EDA)

Característica	Arquitectura Tradicional	Arquitectura Basada en Eventos
Comunicación	Directa, Sincrónica	Indirecta, Asincrónica
Escalabilidad	Limitada	Altamente escalable
Flexibilidad	Fuertemente acoplado	Débilmente acoplado
Perfomance	Ejecución bloqueante	Ejecución no bloqueante
Control de errores	Reintentos síncronos	Reintentos asincrónicos con registros de eventos

- **Procesamiento asíncrono de eventos:** Uso de colas de mensajes (p. ej., RabbitMQ, Kafka) para procesar eventos en paralelo sin bloquear la ejecución.
- **Balanceo de carga:** Distribución del procesamiento de eventos entre múltiples consumidores para evitar la sobrecarga.
- **Particionamiento de eventos:** División de flujos de eventos en particiones más pequeñas para su consumo en paralelo.
- **Agregado y filtrado de eventos:** Reducción del procesamiento redundante mediante la agrupación de eventos o el filtrado de los innecesarios antes de su procesamiento.
- **Escalado automático:** Ajuste dinámico de recursos (p. ej., AWS Lambda, Kubernetes) en función de la carga de eventos.

Fuentes y Tipos de Eventos

- **Eventos de Interface Usuario (UI)**
- **Eventos del Sistema y Hardware**
- **Eventos de Red y E/S**
- **Eventos personalizados** (específicos de la aplicación)

- **Eventos del mouse:** clic, doble clic, pulsar el ratón, subir el ratón, mover el ratón, pasar el ratón por encima, salir del ratón.
- **Eventos del teclado:** pulsar tecla, tecla subir, tecla bajar.
- **Eventos táctiles:** iniciar, mover, finalizar.
- **Eventos de Formulario:** introducir, cambiar, enviar, restablecer.
- **Eventos de Ventana:** redimensionar, desplazar, enfocar, desenfocar.

- **Eventos de Energía:** Inicio, apagado, suspensión o cambios en el estado de la batería del sistema.
- **Eventos del dispositivo:** Conexiones USB, detección de hardware externo o extracción de dispositivos.
- **Eventos de almacenamiento:** Cambios en el sistema de archivos, inserción o extracción de discos.
- **Eventos de proceso:** Monitoreo de la carga de CPU o memoria, creación o finalización de procesos.

- **Eventos de Red:** Establecimiento de conexión, desconexiones, transmisión de datos y tiempos de espera de solicitud.
- **Eventos de E/S:** Creación, modificación, eliminación y lectura/escritura de archivos en el almacenamiento.
- **Eventos de Sockets:** Mensajes de websocket, comunicación servidor-cliente y data streaming.
- **Eventos asíncronos:** Operaciones sin bloqueo que permiten la ejecución en paralelo.

- Creación del Evento
- Emisión del Evento
- Subscripción al Evento
- Despachador del Evento
- Handler del Evento

Frameworks

El término general para una pieza de software que funciona de esta manera (que define puntos de conexión y requiere complementos (plug-ins) es “framework”. Estas son algunas de las definiciones que encontrará si busca “framework” en Google.

Cada definición captura parte del concepto de un framework.

Un framework es:

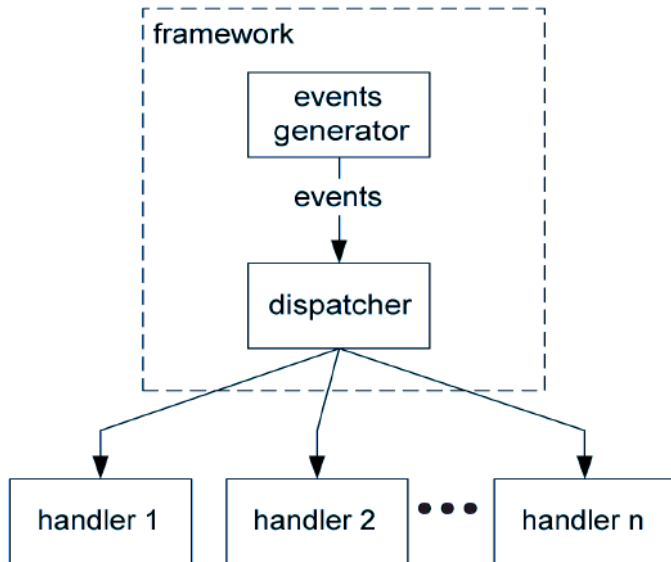
- una estructura básica que soporta o engloba algo más.
- una visión general, esquema o esqueleto, dentro del cual se pueden añadir detalles.
- un entorno de software extensible que se puede adaptar a las necesidades de un dominio específico.
- un conjunto de clases que proporciona una solución general a un problema de aplicación. Un framework suele perfeccionarse para abordar el problema específico mediante la especialización o mediante tipos o clases adicionales.
- Un componente que permite extender su funcionalidad mediante la escritura de módulos complementarios (“extensiones del framework”). El desarrollador de extensiones escribe clases derivadas de las interfaces definidas por el framework.

Un framework es: (continuación)

- Un conjunto de software diseñado para una alta reutilización, con plug-points específicos para la funcionalidad requerida por un sistema en particular. Una vez proporcionados los plug-points, el sistema mostrará un comportamiento centrado en ellos.

El patrón que subyace al concepto de un framework es el patrón Handlers. Las extensiones del framework o complementos son controladores de eventos.

Arquitecturas controladas por eventos - Frameworks



Modelos de Concurrency Basados en Eventos

Concurrencia vs Paralelismo

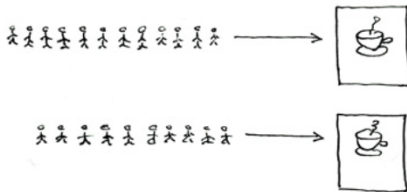
What's the difference between concurrency and parallelism?

Explain it to a five year old.

Concurrent = Two Queues One Coffee Machine



Parallel = Two Queues Two Coffee Machines



© Joe Armstrong 2013

Los modelos de concurrencia basados en eventos definen cómo las aplicaciones gestionan múltiples tareas simultáneamente mientras responden a eventos externos. Estos modelos garantizan un uso eficiente de los recursos y una capacidad de respuesta óptima.

- **Single-Threaded vs. Multi-Threaded Event Processing**
- **Reactive & Proactive Event Handling**
- **Callbacks, Promises, & Async/Await**
- **Cooperative vs. Preemptive Concurrency**

Single-Threaded vs. Multi-Threaded Event Processing

Las aplicaciones basadas en eventos pueden operar en entornos single-threaded o multi-threaded. En un modelo single-threaded, todas las tareas se ejecutan dentro de un único contexto de ejecución, a menudo utilizando un loop de eventos para gestionar operaciones asincrónicas. Este modelo es eficiente para tareas dependientes de E/S, pero puede convertirse en un cuello de botella para operaciones con un uso intensivo de la CPU.

Por otro lado, un modelo multi-threaded permite la ejecución simultánea de múltiples tareas mediante el uso de subprocesos separados. Este enfoque mejora el rendimiento de las cargas de trabajo paralelizables, pero presenta desafíos como condiciones de carrera, deadlocks y sobrecarga de sincronización. La selección del modelo de subprocesos adecuado depende de los requisitos de concurrencia de la aplicación y de las características de la carga de trabajo.

Single-Threaded

Ventajas

- Simplicidad
- Evita condiciones de carrera
- Eficiente para tareas de E/S

Desventajas

- Utilización limitada de la CPU
- Bloqueos retrasan la ejecución

Multi-Threaded

Ventajas

- Mejor utilización de la CPU
- Mayor velocidad en tareas intensivas

Desventajas

- Overhead de sincronización
- Condiciones de carrera y deadlocks

Modelos de Concurrency Basados en Eventos

Factor	Single-Threaded	Multi-Threaded
Mejor para	Tareas limitadas por E/S	Tareas CPU-Intensivas
Complejidad	Baja	Alta
Performance	Eficiente con E/S asincrónicas	Alta para tareas en paralelo
Sincronización	No necesaria	Requerida

Reactive & Proactive Event Handling

Las aplicaciones basadas en eventos pueden emplear estrategias de gestión **reactiva** o **proactiva**. La gestión **reactiva** de eventos implica responder a los eventos a medida que ocurren, sin anticipar interacciones futuras. Este enfoque es común en aplicaciones de interfaz de usuario (UI), servidores de red y sistemas en tiempo real, donde la aplicación espera la entrada y reacciona en consecuencia.

Por el contrario, la gestión **proactiva** de eventos anticipa eventos futuros y prepara respuestas preventivas para optimizar el rendimiento. Esta estrategia se utiliza en análisis predictivo, algoritmos de precarga y sistemas basados en IA.

Ambos enfoques tienen sus ventajas: la gestión reactiva garantiza un uso mínimo de recursos, mientras que la gestión proactiva reduce la latencia de respuesta y mejora la experiencia del usuario.

Reactive Handling

Ventajas

- Eficiente en recursos
- Implementación más simple
- Escalable en Sistemas Asíncronos

Desventajas

- Alta latencia
- Dependencia con Triggers externos

Proactive Handling

Ventajas

- Menor Tiempo de Respuesta
- Experiencia del Usuario Mejorada
- Utilización optimizada de recursos

Desventajas

- Implementación compleja
- Procesamiento potencialmente innecesario

Modelos de Concurrency Basados en Eventos

Factor	Reactive Event Handling	Proactive Event Handling
Tiempo de respuesta	Alta latencia	Baja latencia
Implementación	Simple	Compleja
Mejor para	Interacciones UI, APIs	AI, prefetching, catching
Uso de recursos	Eficiente	Puede ser innecesario

Callbacks, Promises, & Async/Await

La programación basada en eventos se basa en los métodos **Callbacks**, **Promises** y **Async/Await** para gestionar la concurrencia. Las **Callbacks** son funciones que se pasan como argumentos para gestionar eventos de forma asíncrona, comúnmente utilizadas en loops de eventos de JavaScript y Python. Sin embargo, el uso excesivo de **Callbacks** puede provocar un desastre, dificultando la gestión del código.

Las **Promises** simplifican los flujos de trabajo asíncronos al representar un resultado futuro que puede tener éxito o fallar. Permiten el encadenamiento de operaciones, lo que reduce la complejidad.

Async/Await mejora aún más la legibilidad al habilitar una sintaxis similar a la síncrona para el código asíncrono, lo que mejora la capacidad de mantenimiento y la depuración.

Estos mecanismos constituyen la columna vertebral de los sistemas modernos basados en eventos, garantizando la ejecución fluida de operaciones no bloqueantes.

Callbacks

Ventajas

- Simple y efectiva para tareas asincrónicas simples
- Funciona bien con listeners y handlers de eventos.
- Nativo para muchos lenguajes como JavaScript, Python y C.

Desventajas

- Callback Hell
- Problemas de gestión de errores

Promises

Ventajas

- Mayor legibilidad que las Callbacks.
- Gestión de errores integrada con `.catch()`.
- El encadenamiento permite flujos de trabajo asíncronos estructurados.

Desventajas

- Aún requiere estructuras anidadas para tareas dependientes.
- Las promesas deben resolverse o rechazarse explícitamente.

Async/Await

Ventajas

- Elimina la anidación Callbacks y las cadenas de Promises.
- Legibilidad similar a la de un sistema síncrono con mejor gestión de errores.
- Ideal para tareas dependientes de E/S, como solicitudes de red y operaciones con archivos.

Desventajas

- Requiere un loop de eventos (`asyncio.run()`).
- No es adecuado para tareas dependientes de la CPU sin Threads adicionales.

Modelos de Concurrency Basados en Eventos

Factor	Callbacks	Promises	Async/Await
Legibilidad	Baja	Moderada	Alta
Manejo de errores	Complejo	Mejor	Simplificado
Llamadas anidadas	Si	No	No
Mejor Caso de Uso	Eventos simples	Tareas dependientes	Flujos de trabajo asincrónicos complejos

Cooperative vs. Preemptive Concurrency

La concurrencia en la programación basada en eventos puede ser **cooperativa** o **preventiva**. La concurrencia **cooperativa** se basa en que las tareas cedan el control voluntariamente para permitir la ejecución de otras tareas. Este enfoque es común en operaciones de E/S asíncronas, donde un loop de eventos determina el orden de ejecución. Minimiza la sobrecarga por cambio de contexto, pero puede provocar una inanición si las tareas no ceden.

La concurrencia **preventiva**, en cambio, utiliza un scheduler externo para interrumpir forzosamente las tareas y asignar tiempo de CPU. Este modelo se utiliza ampliamente en aplicaciones y sistemas operativos multi-threaded, proporcionando equidad pero aumentando la complejidad de la sincronización. La elección entre estos modelos depende de los requisitos de concurrencia de la aplicación y las limitaciones de rendimiento.

Los modelos de concurrencia basados en eventos determinan la eficiencia con la que las aplicaciones gestionan múltiples tareas.

Cooperative Concurrency

Ventajas

- Bajo sobrecarga (Overhead)
- Ejecución predecible
- Eficiente para aplicaciones con E/S

Desventajas

- Riesgo de bloqueo
- Requiere ceder manualmente

Preemptive Concurrency

Ventajas

- Mejor utilización de la CPU
- Previene la inanición
- Conmutación automática de tareas

Desventajas

- Mayor sobrecarga
- Condiciones de carrera y bloqueos
- Depuración compleja

Modelos de Concurrency Basados en Eventos

Factor	Cooperative Concurrency	Preemptive Concurrency
Conmutación de Tarea	Voluntaria (await, yield)	Forzado por el SO
Sobrecarga	Baja	Alta
Ideal para	Tareas con restricciones de E/S	Tareas con restricciones de CPU
Factor de riesgo	Bloqueo por tareas largas	Condiciones de carrera, bloqueos
Ejemplo de mecanismo	AsynIO, corrutinas	Threads, Multiprocesamiento

¿Preguntas?



FIN

